

# Материалы кафедры ММП факультета ВМК МГУ. Введение в глубокое обучение.

## Занятие 21. Генеративно- сопоставительные сети

Занятие провёл: Денисов Егор (mail:  
denisov.official72@gmail.com)

Материалы составили Денисов Егор (mail:  
denisov.official72@gmail.com), Оганов  
Александр (@wel mud), Находнов Максим

Москва, Весенний семестр 2026

Источники:

- **Generative Adversarial Networks:** [Ian J. Goodfellow et al., 2014](#), примерно 94 тысячи цитирований – работа в которой была предложена идея генеративно сопоставительных сетей
- [Небольшая история о том, как появилась идея GAN](#)
- **DCGAN:** [Alec Radford et al., 2016](#), примерно 22 тысячи цитирований – улучшение GAN модели с помощью выявления принципов обучения и подбора правильной архитектуры; одна из первых моделей качественной генерации цветных изображений разрешением 64 пикселей
- [Обзор на область генеративных моделей от автора DCGAN на конференции ICLR 2026](#) – обзор идей, методов, истории их появлений и науке; **таймкод 35:40**, может потребоваться регистрация
- **Towards Principled Methods for Training Generative Adversarial Networks:** [Martin Arjovsky and Leon Bottou, 2017](#), примерно 3400 цитирований

- **Wasserstein GAN:** [Martin Arjovsky et al., 2017](#), примерно 22 тысячи цитирований – использование расстояния Вассерштейна для оценки схожести распределений (вместо KL дивергенции)
- **WGAN-GP:** [Ishaan Gulrajani et al., 2017](#), примерно 15 тысяч цитирований – появление gradient penalty для соблюдения необходимого условия модели критика
- **R3GAN:** [Yiwen Huang et al., 2025](#), примерно 100 цитат (май 2026) – современный подход к стабильному обучению GAN моделей с высоким разнообразием и теоретическими гарантиями
- **f-GAN:** [Sebastian Nowozin et al., 2016](#), примерно 2300 цитирований
- **About KL Divergences:** [Alan Chan et al., 2022](#), 61 цитирование
- **Generative Trilemma:** [Zhisheng Xiao et al., 2021](#), примерно 900 цитирований
- **How to Train GAN:** [GitHub](#)
- **From GAN to WGAN:** [GitHub](#)
- **Разные реализации:** [GitHub](#)
- **Часть материала основана на курсе "Глубокого обучения" ВМК МГУ**
- **Часть материала основана на семинаре НИУ ВШЭ**
- **Часть материала основана на курсе "Глубокие генеративные модели" AIMasters**

На этом занятии мы впервые затронем понятие генеративных моделей на примере генеративных состязательных сетей (Generative Adversarial Network, GAN). Посмотрим, как одна модель может использоваться при обучении другой, как различные функции потерь влияют на характер обучения, что такое mode collapse, и как его можно избежать. Для этого затронем немного оптимальный транспорт (optimal transport) и будем архитектурно задавать необходимые условия, которые следуют из математики (inductive bias). Ну и наконец поймем, умерли GAN'ы или же область, появившаяся благодаря одному удачному походу в бар, всё ещё жива.

---

## Немного о генеративном моделировании

**Вопрос:** Какие задачи вы уже решали с помощью глубокого обучения? Что их объединяет?

Ответ:

Задачи классификации, сегментации и так далее, формально можно записать как поиск  $p(y \mid x, \theta)$ , где  $y$  метка,  $x$  данные и  $\theta$  параметры модели. Модели, которые находят  $p(y \mid x, \theta)$ , будем называть **дискриминативными** или **дискриминаторами**.

**Вопрос:** Чего нам не хватает для построения полной вероятностной модели?

Ответ:

Если бы мы знали про  $p(x \mid \theta)$ , то смогли бы найти совместное распределение, так как  $p(y \mid x, \theta)p(x \mid \theta) = p(y, x \mid \theta)$ . Поиском и вычислением  $p(x \mid \theta)$ , то есть того как распределены данные, занимается генеративное моделирование. Модели, которые находят  $p(x \mid \theta)$ , будем называть **генеративными**. Сегодня мы впервые поговорим про генеративное моделирование, начнем с постановки задачи.

**Общая постановка задачи:** Пусть есть выборка из неизвестного распределения, цель найти распределение, из которого была получена выборка.

**Вопрос:** Какие модели, решающие задачи генеративного моделирования, вы знаете?

Ответ:

ЕМ-алгоритм.

Когда мы говорим, про глубокое обучение, то очень часто возникают очень общие постановки. Поэтому постараемся уточнить, что мы хотим от модели, и как мы будем ее применять. Будем решать более простую задачу:

**Постановка задачи:** Дана выборка  $\{x_i\}_{i=1}^N$  одинаково распределенных независимых случайных величин,  $x_i \sim q(x_i)$ ,  $q(\cdot)$  – неизвестное распределение. Необходимо найти способ получения случайных величин, которые имеют распределение  $q(\cdot)$ .

Предлагается построить вероятностную модель  $p_\theta(\cdot)$ , из которой можно будет сэмплировать.

## Мотивация и зачем нужно знать распределения

Представим у нас есть возможность сэмплировать из  $p(x \mid \theta)$ , что это нам дает? Если данные это изображения котиков, то мы могли бы получать новые изображения котиков. Если у нас данные это молекулы определенного класса, то мы смогли бы находить новые молекулы и так далее.

**Вопрос:** Как вы думаете, что мы ожидаем от изображений котиков?

Ответ:

Ответ на этот вопрос дает **генеративная триллема**, которая ничего не доказывает и не утверждает, а просто описывает три основных требования к генеративным моделям. Предполагается, что сэмплы, полученные из "хорошей" вероятностной модели, будут:

- Качественные, то есть получаемые случайные величины будут иметь распределение очень похожее на  $q(\cdot)$ ;
- Разнообразные, то есть будут покрываться разные моды;
- Вычислительно эффективные.

В области глубокого обучения все эти требования проверяются **экспериментально**, так как многие понятия неформализуемы. Например, мы не можем описать математически хорошее изображение котика. Пока не существует подхода, который предложит вероятностную модель, удовлетворяющую всем требованиям.

## Генеративные состязательные сети, Generative adversarial networks (GAN)

Основной рассказ будет следовать работе [Ian J. Goodfellow et al., 2014](#), примерно 94 тысяч цитат.

Наша цель – описать генеративную модель и понять, как ее использовать. Для этого мы будем использовать генеративно-состязательный подход.

Представим такую ситуацию: есть мошенник, и его цель – научиться подделывать деньги (получать сэмплы из распределения данных). Как он может этому научиться? Он может постараться подделать купюры хоть как-то и попробовать обмануть банк. Пусть в банке есть очень добрый инспектор, который проверит купюры, выделит основные моменты, по которым можно классифицировать фальшивку, и скажет мошеннику, где он ошибся. После чего мошенник научится и исправит ошибки. Повторяя такой подход снова и снова, он в конце концов сможет идеально обманывать инспектора банка. [Источник аналогии](#). Попробуем этот подход записать математически.

Мы работаем в предположении, что мы всегда можем обучить идеальную дискриминативную модель (классификатор), которую будем обозначать  $D(x)$ .

Пусть у нас есть какая-то нейронная сеть  $G(z; \theta_g)$  с параметрами  $\theta_g$ , которую будем называть генератором. Цель генератора – получить такие объекты, которые наш идеальный классификатор считал бы настоящими. При этом генератор не может делать объекты из воздуха, ему на вход нужно что-то передать. Самое легкое решение – передавать что-то максимально простое, поэтому будет считать, что  $z$  – это

случайная величина, которая сэмплируется из простого распределения, например,  $\mathcal{N}(0, I)$ . Математически задачу генератора можно записать так:

$$\min_G \mathbb{E}_{z \sim p(z)} \log(1 - D(G(z; \theta_g))).$$

Казалось бы, вот получили мы задачу минимизации, решим ее и все готово, но...

**Вопрос:** А действительно ли мы решаем такую задачу? Что значит идеальный дискриминатор?

Ответ:

После того, как мы поучим наш генератор, и он научится обманывать дискриминатор хотя бы как-то, наш дискриминатор уже не будет идеальным, поэтому его тоже нужно будет учить. Простая аналогия – если вы превзошли своего учителя, то это может означать не то, что вы лучший, а то, что вам нужен другой учитель...

Будем считать, что идеальный классификатор идеально отличает сгенерированный объект от настоящего, то есть является решением задачи:

$$\max_D \mathbb{E}_{x \sim p_{data}(x)} \log D(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D(G(z; \theta_g))).$$

Чем-то напоминает минимизацию кросс энтропии (вернемся к этому позже)...

Тем самым мы получили следующую задачу:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}(x)} \log D(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D(G(z; \theta_g))).$$

**Вопрос:** Как называются такие задачи?

Ответ:

Такие задачи называются седловыми, они еще встретятся вам в курсе теории игр.

**Вопрос:** В чем главная проблема в обучении?

Ответ:

Седловые задачи сложные :( А еще требуют хорошего решения внутренней задачи оптимизации дискриминатора.

## Идеальные дискриминаторы и генераторы

Наша цель – понять, как будет выглядеть обучение генератора: какие существуют

проблемы, какие есть нестабильности и так далее. Чтобы все это понять, попробуем увидеть, как он выглядит математически в явном виде. Для этого стоит определиться с тем, что такое идеальный дискриминатор.

Мы задали  $G(\cdot; \theta_g)$  как нейросеть, а следовательно мы получаем, что генератор  $G$  индуцирует какое-то распределение (так как  $z \sim p(z)$ , а значит  $x = G(z)$  – функция от случайной величины, то есть тоже случайная величина). Будем называть распределение генератора  $p_g(x)$  (при желании это распределение можно даже посчитать по-честному, используя теорему о замене переменной), а наша цель добиться того, чтобы  $p_g(x) = p_{data}(x)$ . Сначала нам надо понять, как выглядит идеальный классификатор для фиксированного генератора  $G$ . Для этого нам надо посчитать градиент кросс-энтропии и приравнять его к нулю. Получим:

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}.$$

**Вопрос:** Как будет выглядеть выражение если  $p_{data} = p_G$ ?

Ответ:

Идеальный дискриминатор в таком случае для любого объекта будет выдавать вероятность  $p = \frac{1}{2}$ , что вполне соотносится с нашими ожиданиями.

Выходит, что при наличии идеального дискриминатора для генератора мы получили следующую оптимизационную задачу для генератора:

$$\min_G \mathbb{E}_{x \sim p_{data}(x)} \log \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} + \mathbb{E}_{x \sim p_g(x)} \log \frac{p_g(x)}{p_{data}(x) + p_g(x)}.$$

Выглядит как-то знакомо...

**Вопрос:** Чему равен минимум выражения при идеальном генераторе ( $p_g = p_{data}$ )?

Ответ:

$-\log 4$

**Вопрос:** На сумму каких стандартных функционалов вам напоминает полученное выражение?

**Подсказка:** Когда мы говорим о распределениях, стоит задуматься о дивергенциях.

Ответ:

О чудо, это же KL-дивергенции. Здесь можно подушнить и сказать, что сумма двух плотностей не будет плотностью, так как проинтегрируется в двоичку, но мы можем спокойно исправить это, поделив сумму на 2, что приведет нас к следующей оптимизационной задаче.

На самом деле при наличии идеального классификатора мы получим, что для поиска генератора нужно решить задачу:

$$-\log 4 + KL(p_{data} \parallel \frac{p_{data} + p_g}{2}) + KL(p_g \parallel \frac{p_{data} + p_g}{2}) \rightarrow \min_G.$$

Если мы бы знали все дивергенции на свете, то смогли бы увидеть дивергенцию Йенсена-Шеннона, которая определяется как:

$$JSD(p_{data} \parallel p_g) = \frac{1}{2} KL(p_{data} \parallel \frac{p_{data} + p_g}{2}) + \frac{1}{2} KL(p_g \parallel \frac{p_{data} + p_g}{2}),$$

то есть, на самом деле, для генератора мы решаем задачу:

$$-\log 4 + 2JSD(p_{data} \parallel p_g) \rightarrow \min_G.$$

**Вопрос:** Какие возникнут проблемы при решении этой задачи на практике?

Ответ:

Об этом будет рассказано в следующей секции.

## Чем плоха дивергенция?

Представим, что мы хотим приблизить распределение  $p$ , в котором есть две моды (например смесь из двух нормальных распределений), с помощью распределения  $q$ . Мы будем искать  $q$  в классе нормальных распределений, настраиваемыми параметрами будут математическое ожидание и дисперсия. Похожая ситуация познается на практике, так как распределение данных очень сложное и не всегда семейство нейросетей достаточно богатое, чтобы покрыть все моды.

## Forward KL

Представим, что мы решаем задачу  $KL(p \parallel q) \rightarrow \min_q$ , то есть  $-\mathbb{E}_{x \sim p(x)} \log q(x) \rightarrow \min_q$ . Такая KL дивергенция называется **прямой (forward)**.

Чему будут равны оптимальные параметры при такой оптимизации? Мы хотим, чтобы у объектов  $x$ , полученных из  $p(\cdot)$ , была как можно больше вероятность  $q(x)$ . Тем самым мы должны покрыть все возможные  $x$ , получаемые из  $p(x)$ , и дать им как можно большую вероятность.

На практике мы получаем эффект, который называется **mean-seeking**, то есть мы ищем параметры распределения  $q$  так, чтобы распределение  $q$  покрыло все распределение  $p$  "в среднем хорошо".

## Reverse KL

Если мы решаем задачу  $KL(q||p) \rightarrow \min_q$ , то есть

$\mathbb{E}_{x \sim q(x)} \log q(x) - \mathbb{E}_{x \sim q(x)} \log p(x) \rightarrow \min_q$ . Такая KL дивергенция называется **обратной (reverse)**.

Чему будут равны оптимальные параметры при такой оптимизации? Мы

минимизируем  $\mathbb{E}_{x \sim q(x)} \left[ \log q(x) - \log p(x) \right]$ , то есть на сэмплах из  $q(\cdot)$  вероятности  $p(x)$  и  $q(x)$  должны быть как можно более схожи. Заметим, что  $\mathbb{E}_{x \sim q(x)} \log q(x)$  – энтропия распределения  $q(x)$ , и некоторые свойства можно получить из этого.

На практике мы получаем эффект, который называется **mode-seeking**, то есть мы выбираем параметры распределения  $q$  так, чтобы оно наиболее совпадало с распределением  $p$  в какой-то "определенной зоне". Например, найденные параметры могут приблизить только одну из мод.

Подробнее о проблемах мы поговорим на семинаре. Похожие проблемы имеет и дивергенция Йенсена-Шеннона, то есть проблема с разнообразием мод возникает в **GAN** моделях на уровне функционала и следует из теории.

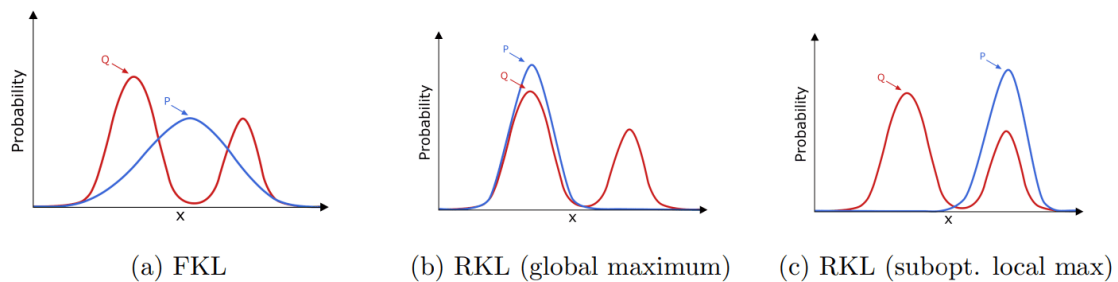


Figure 2: Mean-seeking (FKL) and mode-seeking (RKL) behavior.

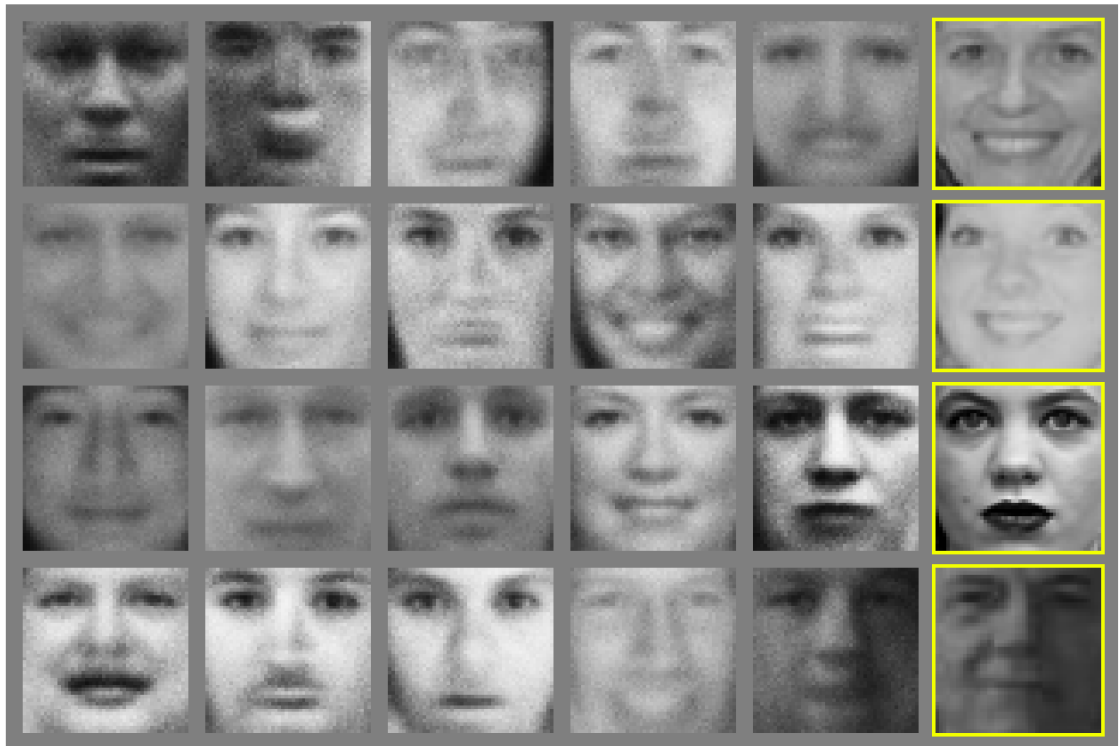
*RKL и FKL обозначают reverse KL и forward KL соответственно. В данном примере учат параметры распределения  $p$ .* [Источник](#).

## Проблемы

Как мы видели выше, чаще всего мы получаем данные из одной моды, что является следствием нашего функционала. После такого печального результата мы можем задуматься, а действительно ли **GAN** модели обучаемы?

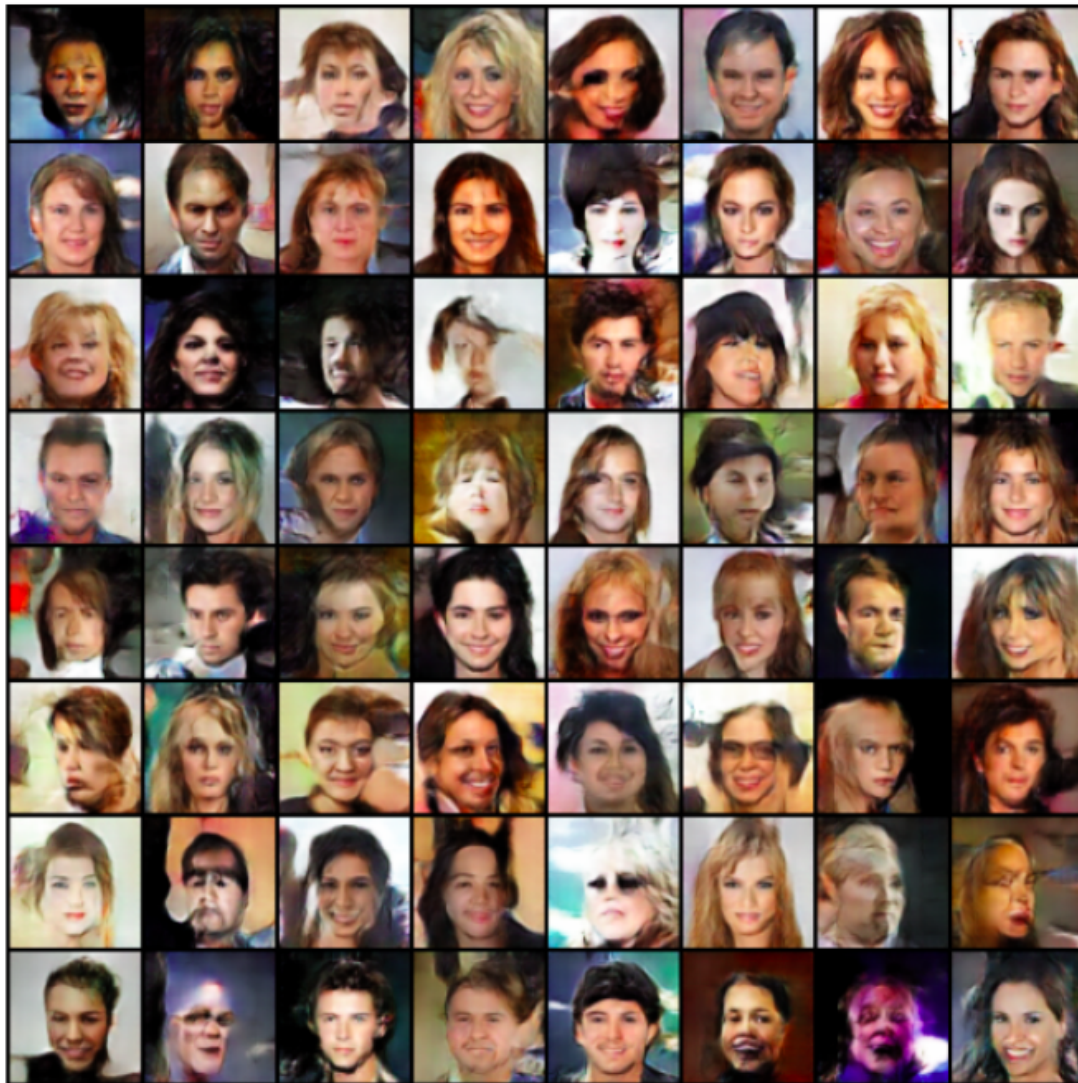
Обучаемы, но есть нюанс!

Ниже представлены примеры сгенерированных изображений из оригинальной статьи (2014 год):



[Источник](#)

[DCGAN с кодом](#) с реализацией на PyTorch, получает такие сэмплы (2016 год):



#### Источник

Прошло 2 года, при этом качество генерации заметно возросло, что же поменялось за это время?

1. Люди перешли к сверточным нейронным сетям и стали использовать модели с большим числом параметров;
2. Научились правильнее настраивать параметры модели и архитектуры.

Но самое важное, придумал кучу разных лайфхаков, как учить GAN. Примеры: [ОБЯЗАТЕЛЬНО К ПРОЧТЕНИЮ](#), [менее ОБЯЗАТЕЛЬНО К ПРОЧТЕНИЮ](#)

К счастью, ни одними инженерными лайфхаками мы едины, поэтому в 2017 году был предложена модель Wasserstein GAN или кратко – WGAN [M Arjovsky et al., 2017](#), примерно 22 тысячи цитат.

# GAN и математика

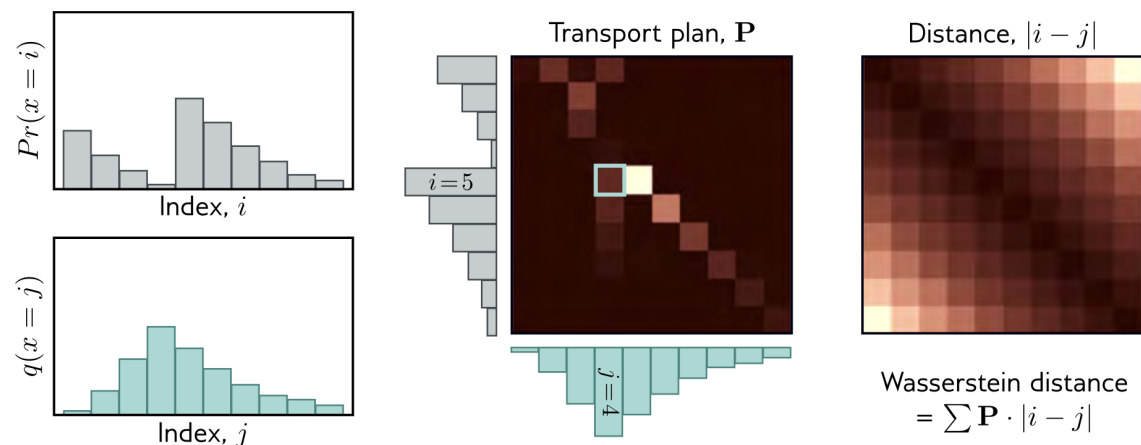
Выше мы показали, что проблема мод коллапса, может быть, следствием вида функционала для обучения генератора. Откуда возникает идея: а как мы можем поменять функцию потерь, но сохранить идею модели?

Для этого ненадолго заглянем в раздел математики, под названием **оптимальный транспорт**.

## Оптимальный транспорт

### Расстояние Вассерштейна (дискретное)

**Неформальное определение:** Минимальная стоимость перемещения и преобразования кучи грунта в форме одного распределения вероятности в форму другого распределения.



[Источник](#)

### Расстояние Вассерштейна (непрерывное)

$$W(\pi, p) = \inf_{\gamma \in \Gamma(\pi, p)} \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim \gamma} \|\mathbf{x} - \mathbf{y}\| = \inf_{\gamma \in \Gamma(\pi, p)} \int \|\mathbf{x} - \mathbf{y}\| \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y}.$$

$\gamma(\mathbf{x}, \mathbf{y})$  – транспортный план (количество "грунта" которое нужно сместить с точки  $\mathbf{x}$  в точку  $\mathbf{y}$ )

$$\int \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} = p(\mathbf{y}); \quad \int \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{y} = \pi(\mathbf{x}).$$

$\Gamma(\pi, p)$  – множество всех возможных совместных распределений  $\gamma(\mathbf{x}, \mathbf{y})$  with marginals  $\pi$  and  $p$ .

$\gamma(\mathbf{x}, \mathbf{y})$  – масса;

$\|\mathbf{x} - \mathbf{y}\|$  – расстояние.

Мы будем использовать  $W(\pi, p)$  для задачи оптимального транспорта (есть разные варианты метрики Вассерштейна, как и разные варианты  $L_p$  норм).

Можно доказать, что для любой последовательности  $\{p_t\}_{t=1}^\infty$  на компакте, такой что верно  $D_{KL}(\pi \| p_t) \rightarrow 0$ , при  $t \rightarrow \infty$  будет верно:

$$W(\pi, p_t) \rightarrow 0, \quad t \rightarrow \infty.$$

Теорема (двойственность Кантаровича-Рубенштейна)

$$W(\pi \| p) = \frac{1}{K} \sup_{\|f\|_L \leq K} [\mathbb{E}_{\pi(\mathbf{x})} f(\mathbf{x}) - \mathbb{E}_{p(\mathbf{x})} f(\mathbf{x})],$$

где  $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ,  $\|f\|_L \leq K$ , то есть  $f$  является  $K$ -Липшицева непрерывная функция ( $f: \mathcal{X} \rightarrow \mathbb{R}$ ). Заметим, что выражение  $W(\pi \| p)$  будет одинаковым при любом выборе  $K$ , так как, если  $\|f\|_L \leq K_1$ , то можем записать

$f = \frac{K_1}{K_2} g$ ,  $\|g\|_L \leq K_2 \Rightarrow$  получим эквивалентность выражений для любых  $K$ .

Какой смысл в этой формуле? Если у нас распределения одинаковые, то математические ожидания по любой функции  $f$  будут равны. Если распределения где-нибудь отличаются, то мы будем подбирать такие  $f$ , которые больше выделяют эту разницу.

**Вопрос:** Зачем мы требуем липшицевость функций?

Ответ:

Липшицевость является ограничением на рост функции, тем самым мы ограничиваем расстояние Вассерштейна и рассматриваем только разумные функции (без дельта функций и тд).

**Замечание:** Мы можем заменить  $\sup$  на  $\max$ , так как мы предполагаем компактность вероятностных мер, а по теореме Арцела множество непрерывных функций на компакте  $X$  липшицевых с константой  $\leq K$  компактно в  $C(X)$ . Значит  $\sup$  достигается на элементе множества, а значит его можно заменить на  $\max$ .

Осталось только научиться минимизировать  $W(\pi \| p)$  – например, использовать Монте-Карло оценку, правда?...

К сожалению нет, нужно еще научиться задавать Липшицевы функции с помощью нейронной сети...

**Вопрос:** Как влиять на рост функции, которая задана нейронной сетью?

Ответ:

Если веса модели находятся в каком-нибудь компакте, то чаще всего этого будет достаточно для ограничения роста функции.

Теперь нам нужно убедиться, что  $f$  является  $K$ -непрерывной Липшицевой функцией.

Пусть  $f_\phi(\mathbf{x})$  – нейронная сеть, параметризованная  $\phi$ . Если параметры  $\phi$  лежат в компакте  $\Phi$ , то  $f_\phi(\mathbf{x})$  будет  $K$ -непрерывной Липшицевой функцией.

Пусть параметры будут привязаны к фиксированному компакту  $\Phi \in [-c, c]^d$  (например,  $c = 0.01$ ) после каждого обновления градиента.

$$\begin{aligned} K \cdot W(\pi \| p) &= \max_{\|f\|_L \leq K} [\mathbb{E}_{\pi(\mathbf{x})} f(\mathbf{x}) - \mathbb{E}_{p(\mathbf{x})} f(\mathbf{x})] \geq \\ &\geq \max_{\phi \in \Phi} [\mathbb{E}_{\pi(\mathbf{x})} f_\phi(\mathbf{x}) - \mathbb{E}_{p(\mathbf{x})} f_\phi(\mathbf{x})] . \end{aligned}$$

## Необходимое условие решения

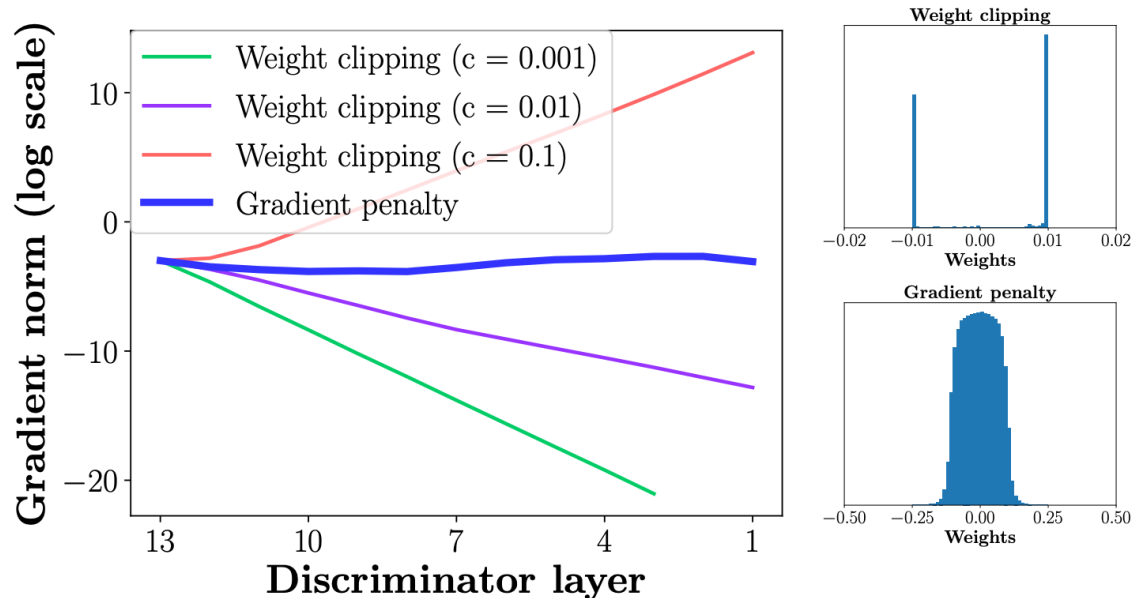
Зачастую достаточные условия являются слишком громоздкими и сложными для того, чтобы закладывать их в модель. Однако, нередко бывает, что достаточно только необходимых. Оказывается, если функция  $f$  – 1-Липшицева дифференцируемая функция, кроме того,  $f$  является решением задачи оптимального транспорта в смысле двойственности Кантаровича-Рубенштейна, то будет верно следующее:

$$\mathbb{P}_{(\mathbf{y}, \mathbf{z}) \sim \gamma} \left[ \nabla f(\hat{\mathbf{x}}_t) = \frac{\mathbf{z} - \hat{\mathbf{x}}_t}{\|\mathbf{z} - \hat{\mathbf{x}}_t\|} \right] = 1,$$

где  $\hat{\mathbf{x}}_t = t\mathbf{y} + (1 - t)\mathbf{z}$ .

В частности нам важно, что почти наверно  $\nabla f(\hat{\mathbf{x}}_t) = 1$ , будем это требовать у нейронной сети (то есть штрафовать за отклонения нормы градиента от 1)

Тем самым получим **WGAN** с gradient penalty **WGAN - GP** [Ishaan Gulrajani et al., 2017](#), примерно 13 тысяч цитат.



Источник

## Итоги

### WGAN

**WGAN** модель использует клиппинг весов для достижения Липшицевости функции критика.

Функция потерь

$$\min_G W(\pi||p) \approx \min_G \max_{\phi \in \Phi} [\mathbb{E}_{\pi(\mathbf{x})} f(\mathbf{x}, \phi) - \mathbb{E}_{p(\mathbf{z})} f(G(\mathbf{z}, \theta), \phi)] .$$

Здесь  $f(\mathbf{x}, \phi)$  модель критика. Веса критика  $\phi$  должны принадлежать компакту  $\Phi = [-c, c]^d$ .

### WGAN-GP

**WGAN-GP** модель использует gradient penalty для достижения Липшицевости функции критика

Функция потерь

$$W(\pi||p) = \underbrace{\mathbb{E}_{\pi(\mathbf{x})} f(\mathbf{x}) - \mathbb{E}_{p(\mathbf{x}|\theta)} f(\mathbf{x})}_{\text{original critic loss}} + \underbrace{\lambda \mathbb{E}_{U[0,1]} \left[ (\|\nabla_{\hat{\mathbf{x}}} f(\hat{\mathbf{x}})\|_2 - 1)^2 \right]}_{\text{gradient penalty}} ,$$

где точки  $\hat{\mathbf{x}}_t = t\mathbf{x} + (1-t)\mathbf{y}$  с  $t \in [0, 1]$  равномерно засэмплированы вдоль прямой соединяющей точки:  $\mathbf{x}$  из распределения данных  $\pi(\mathbf{x})$  и  $\mathbf{y}$  распределения генератора  $p(\mathbf{x} | \theta)$ .

## f-дивергенция

Выше мы выводили  $GAN$  для  $JSD$ , теперь для расстояния Вассерштейна, а может ли быть так, что это все частные записи какой-то дивергенции записанной в общем виде?

Да, существует понятие  $f$ -дивергенции, мы не будем вводить строгое определение, но при некоторых условиях на функцию  $f$ , мы можем получить, что многие дивергенции можно записать как  $D_f(P\|Q) := \mathbb{E}_{p(x)} f\left(\frac{p}{q}\right)$ . Пример  $GAN$  с разными функциями  $f$  представлены в статье [f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization](#), [S Nowozin et al., 2016](#), примерно 2300 цитат.

| Name              | $D_f(P\ Q)$                                                                                 | Generator $f(u)$                       |
|-------------------|---------------------------------------------------------------------------------------------|----------------------------------------|
| Kullback-Leibler  | $\int p(x) \log \frac{p(x)}{q(x)} dx$                                                       | $u \log u$                             |
| Reverse KL        | $\int q(x) \log \frac{q(x)}{p(x)} dx$                                                       | $-\log u$                              |
| Pearson $\chi^2$  | $\int \frac{(q(x)-p(x))^2}{p(x)} dx$                                                        | $(u-1)^2$                              |
| Squared Hellinger | $\int \left(\sqrt{p(x)} - \sqrt{q(x)}\right)^2 dx$                                          | $(\sqrt{u}-1)^2$                       |
| Jensen-Shannon    | $\frac{1}{2} \int p(x) \log \frac{2p(x)}{p(x)+q(x)} + q(x) \log \frac{2q(x)}{p(x)+q(x)} dx$ | $-(u+1) \log \frac{1+u}{2} + u \log u$ |
| GAN               | $\int p(x) \log \frac{2p(x)}{p(x)+q(x)} + q(x) \log \frac{2q(x)}{p(x)+q(x)} dx - \log(4)$   | $u \log u - (u+1) \log(u+1)$           |

[Источник](#)

## А что сейчас происходит с GAN'ами?

Несмотря на то, что сейчас в задачах генерации модным направлением являются диффузионки, область GAN'ов всё еще исследуется. Так, [Yiwen Huang et al., 2025](#) говорят о том, что сложное обучение GAN'ов – это миф, и предлагают новый, теоретически обоснованный бейзлайн – [R3GAN](#).

Стандартная функция потерь:

$$\mathcal{L}(\theta, \psi) = \mathbb{E}_{z \sim p_z} [f(D_\psi(G_\theta(z)))] + \mathbb{E}_{x \sim p_D} [f(-D_\psi(x))].$$

Функция потерь R3GAN:

$$\mathcal{L}(\theta, \psi) = \underbrace{\mathbb{E}_{z \sim p_z, x \sim p_D} [f(D_\psi(G_\theta(z)) - D_\psi(x))]}_{\text{RpGAN}} + \underbrace{\frac{\gamma}{2} \mathbb{E}_{x \sim p_D} [\|\nabla_x D_\psi\|^2]}_{\text{R1}} + \underbrace{\frac{\gamma}{2} \mathbb{E}_{x \sim p_\theta} [f(D_\psi(x))]}_{\text{R2}}$$

**Интуиция:** Дискриминатор теперь оценивает, насколько генерируемый образец  $G_\theta(z)$  реален относительно конкретного реального  $x$ . Таким образом, для оптимизации лосса недостаточно научиться генерировать из одной моды реального распределения, потому что тогда в среднем относительная ошибка будет больше, чем в случае, когда

генератор покроем все моды исходного распределения. Слагаемые  $R_1$ ,  $R_2$  выступают в качестве регуляризации для дискриминатора.

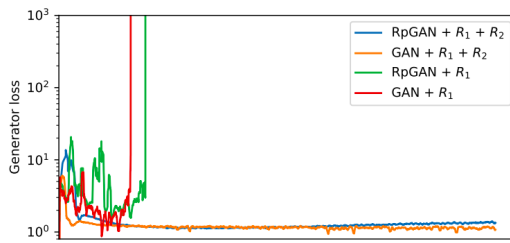


Figure 1: Generator  $G$  loss for different objectives over training. Regardless of which objective is used, training diverges with only  $R_1$  and succeeded with both  $R_1$  and  $R_2$ . Convergence failure with only  $R_1$  was noted by Lee et al. [42].

| Loss                  | # modes $\uparrow$ | $D_{KL}\downarrow$ |
|-----------------------|--------------------|--------------------|
| RpGAN + $R_1$ + $R_2$ | <b>1000</b>        | <b>0.0781</b>      |
| GAN + $R_1$ + $R_2$   | 693                | 0.9270             |
| RpGAN + $R_1$         | Fail               | Fail               |
| GAN + $R_1$           | Fail               | Fail               |

Table 1: StackedMNIST [46] result for each loss function. The maximum possible mode coverage is 1000. “Fail” indicates that training diverged early on.

Источник

## Выводы

1. Мы впервые обучали одну нейронную сеть через другую;
2. Мы не можем определить метрики, которые оценивают близость между распределениями изображений, про которые мы ничего не знаем. Однако, мы можем обучить дискриминатор и использовать его в качестве некоторой метрики, по которой будем оптимизировать;
3. Есть свобода в выборе обучаемой метрики и можно переформулировать задачу в минимизацию расстояния Вассерштейна;
4. Использование необходимых условий может помочь в обучение модели. Раньше мы закладывали знания о данных в архитектуру (сверточные сети, rnn, трансформеры), теперь мы использовали знания в построении функции потерь.
5. Область всё еще жива, и даже в 2025 году люди находят в ней что-то новое.